UCLouvain

LINGI2364

Mining Patterns in Data

Project 1 : Implementing Apriori

Group 13

Kha Luân Feltus - 79982400
Siénou Lamien - 72642301



ÉCOLE POLYTECHNIQUE DE LOUVAIN

Academic year 2024-2025

Table des matières

1 Introduction **1**

2 Apriori algorithm **1**
2.1 Apriori implementation 1
2.2 Optimisation implementation 2
2.2.1 Trie 2
2.2.2 Apriori principle 2

3 Depth First Search algorithm **2**
3.1 Depth First Search implementation 2
3.2 Optimisation implementation 3
3.2.1 Trie 3
3.2.2 Vertical representation 3
3.2.3 Projection 3
3.2.4 Sorted Order 3

4 Experimental Results Analysis **3**
4.1 Methodology 3
4.2 Time Complexity Analysis 3
4.3 Memory Usage Analysis 4

5 Conclusion **4**

1 Introduction

Among association mining techniques, the Apriori and Eclat algorithms occupy a central place for the discovery of frequent patterns in transaction sets. Apriori, introduced by Agrawal and Srikant, is based on an iterative approach and a breadth-first exploration of the space of frequent item sets, while Eclat, proposed by Mohammed Zaki, uses a vertical representation of the data and a depth-first exploration.
The aim of this report is to compare these two algorithms in terms of operation, efficiency and applicability, in order to highlight their respective advantages and limitations.

2 Apriori algorithm

The Apriori algorithm is a data mining algorithm used to discover association rules in large datasets. This algorithm is based on a key assumption : A frequent subset of a frequent item set is also frequent.

2.1 Apriori implementation

The function 1 counts the occurrences of individual items in the dataset by iterating through each transaction and updating the dictionary if the candidate item is a subset of a transaction. It then filters out the items that meet the minimum support threshold to form the initial set of frequent item sets. The function proceeds to insert these frequent item sets into the trie with the minimum frequency. It then enters a loop to generate and count candidate item sets of increasing size. In each iteration, it prunes the candidate item sets and counts their occurrences in the dataset. The frequent candidates that meet the minimum count threshold are inserted into the trie, and the loop continues with the new set of frequent candidates. The process repeats until no more frequent candidates are found. Finally, the function prints the trie structure.

```
def apriori(filepath, minFrequency):
    """Runs the apriori algorithm on the specified file with the given minimum frequency""
    "
    dataset = Dataset(filepath)
    min_count = minFrequency * dataset.trans_num()
    root = Node(0,None,0)
    item_counts = defaultdict(int)

    for transaction in dataset.transactions:
        for item in transaction:
            item_counts[(item,)] += 1
    frequent_itemsets = {item: count for item, count in item_counts.items() if count >=
    min_count}
    for item, count in frequent_itemsets.items():
        insert(root,item, count / dataset.trans_num())
    k = 2

    current_level = list(frequent_itemsets.keys())
    while current_level:
        candidates = prune_candidate(current_level)
        candidate_counts = count_candidates(candidates, dataset.transactions)
        frequent_candidates = {item: count for item, count in candidate_counts.items() if
    count >= min_count}
        if not frequent_candidates:
            break
        for item, count in frequent_candidates.items():
            insert(root,list(item), count / dataset.trans_num())
        current_level = list(frequent_candidates.keys())
        k += 1

    print_trie(root)
    return root
```

Listing 1 – Apriori algorithm

2.2 Optimisation implementation

2.2.1 Trie

We use a trie data structure to store the frequent item sets. The trie is a tree-like data structure in which each node represents a string and the child nodes represent the characters of the string. In our case, the trie is used to store the frequent item sets and their respective minimum support frequency. The trie gives us some advantages properties :

— The memory economy that avoid the redundancy of common prefixes between frequent patterns.

— Searching for a frequent item set is quick, because you can browse the tree by following the common prefix.

— Insertion and search complexity are O(k) (where k is the length of the pattern).

2.2.2 Apriori principle

We use the principle of the Apriori algorithm to prune candidate elements. First of all, to create a new candidate, we go through the previous frequent elements. We check whether any of them have the same common prefix. If so, we merge the two elements to create a new candidate. We then check whether the new candidate is frequent.

3 Depth First Search algorithm

The Eclat algorithm is an efficient method for frequent item set mining, similar to Apriori, but it operates in a different way. Instead of using a breadth-first search and candidate generation like Apriori, Eclat relies on a depth-first search and vertical database format, making it faster in many cases.

3.1 Depth First Search implementation

The function 2 converts the dataset into a vertical database representation, where each item is associated with the list of transaction IDs in which it appears. An empty cache dictionary is also initialized to store intermediate results. Then the function sorts the items in the vertical database by the length of their transaction ID lists in descending order. It then iterates through each item and its associated transaction IDs. For each item, it creates a new item set containing just that item. If the length of the transaction ID list for the item is greater than or equal to the minimum support count, the item set is inserted into the trie with its minimum support frequency. The function then generates a new vertical database by projecting the original vertical database on the current item. This projection reduces the database to only include transactions that contain the current item. The DFS function is called to recursively explore and count larger item sets that include the current item, using the new vertical database. Finally, the function prints the trie structure.

```
def alternative_miner(filepath, minFrequency):
    """Runs the alternative frequent itemset mining algorithm on the specified file with the given minimum frequency"""
    dataset = Dataset(filepath)
    trans_num = dataset.trans_num()
    root = Node(0,None,0)

    min_support = minFrequency * trans_num
    full_vertical_db = vertical_representation(dataset)
    cache = {}
    sorted_items = sorted(full_vertical_db.items(), key=lambda x: -len(x[1]))
    for key, tids in sorted_items:
        new_itemset = [key]
        if len(tids) >= min_support:
            insert(root,new_itemset, len(tids) / trans_num)
            cache[frozenset(new_itemset)] = new_itemset
            new_vdb = projection(full_vertical_db, key, min_support)
            dfs(root,new_itemset, new_vdb, min_support, cache, trans_num)

    print_trie(root)
    return root
```

Listing 2 – Eclat algorithm

3.2 Optimisation implementation

3.2.1 Trie

A trie is also used for the Eclat algorithm in the same way as Apriori, with the same advantages.

3.2.2 Vertical representation

Eclat stores each item with the list of transactions (TID-list) in which it appears. This representation give some advantages :

- **Avoids repeated database scans** : The transactional database is no longer scanned after the initial conversion.
- **Operations based on intersections** : Generating frequent item sets is like doing list intersections instead of checking each transaction individually.
- **Less storage for dense databases** : For databases where many items coexist in transactions, this approach saves memory.

3.2.3 Projection

Instead of storing all the complete TID-lists, Eclat projects the lists as you explore them that gives interesting properties.

- **Exponential reduction in memory space for each new intersection** : A smaller version of the TID-list is created instead of storing the entire initial database.
- **Optimisation of recursive traversal** : Only relevant transactions are kept at each stage.
- **Avoids unnecessary processing of obsolete data** : Each subset is explored in a subset of transactions, speeding up processing.

3.2.4 Sorted Order

In the Eclat algorithm, sorting items in descending order of support offers several key advantages in terms of performance and optimisation. By starting with the most frequent items, you can quickly generate interesting candidates and avoid exploring unpromising branches unnecessarily. By sorting from the most frequent to the least frequent, pruning can be applied more effectively. The most frequent items have longer TID-lists, but their intersections are often quicker and more relevant than those of rare items.

4 Experimental Results Analysis

4.1 Methodology

To conduct the experimental comparison, we performed each test 10 times to ensure reliable results. We then calculated the average time and memory usage for each test. To collect these metrics, we utilized the `time` and `traceMalloc` libraries, which provided accurate records of execution time and memory allocation. All experiments were conducted on an Apple M1 device.

4.2 Time Complexity Analysis

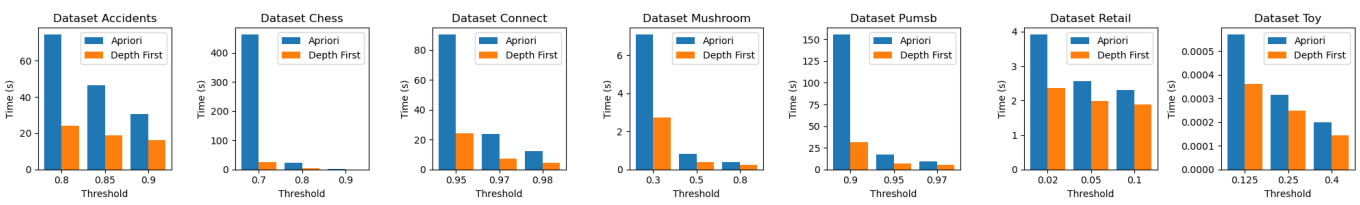


FIGURE 1 – Time analysis

The time complexity results aligns with our theoretical expectations :

— **Algorithm Comparison** : The Depth First algorithm consistently outperforms the Apriori algorithm in terms of execution time across all datasets and thresholds. This significant performance difference is particularly pronounced in larger datasets like Chess, Connect, and Pumsb, where Apriori takes several times longer to execute. The Depth First approach's efficiency comes from its ability to quickly mine patterns without generating all candidate sets in memory.

— **Threshold Impact** : For both algorithms, as the support threshold increases, the execution time decreases. This is expected behavior since higher thresholds result in fewer frequent itemsets that need to be generated and processed, reducing the computational load.

— **Dataset Size Influence** : The execution time scales with dataset complexity and size. Larger and more complex datasets (Pumsb, Connect, Chess) require significantly more processing time compared to smaller datasets (Toy), particularly for the Apriori algorithm.

4.3 Memory Usage Analysis

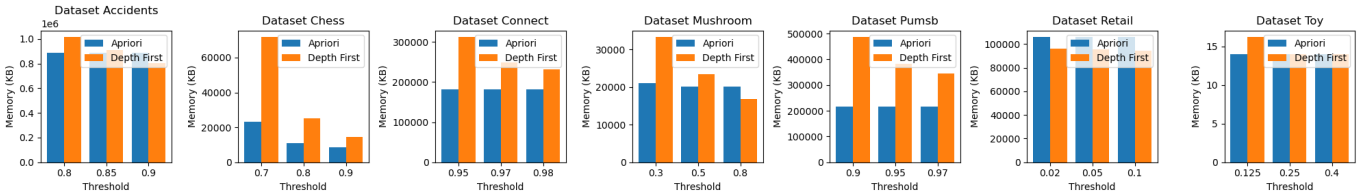


FIGURE 2 – Memory analysis

The memory usage results show some unexpected patterns :

— **Anomalous Results** : Contrary to theoretical expectations, the Depth First algorithm consistently shows higher memory usage than Apriori across most datasets and thresholds. Theoretically, DFS should utilize less memory than Apriori algorithm. These anomalous results may be due to the caching mechanism implemented in our Depth First algorithm. The caching system stores previously computed results to avoid redundant computations, which improves time efficiency but adds memory overhead. In the first execution, the cache requires substantial memory allocation, while subsequent executions might reuse this cache, creating misleading measurements. This caching effect likely distorts the true memory efficiency comparison between the two algorithms.

— **Threshold Impact** : For both algorithms, higher support thresholds generally result in lower memory usage, which is expected as fewer patterns need to be stored.

5 Conclusion

Although Apriori is a simple and intuitive approach, it nevertheless suffers from its widthwise exploration, which can be costly in terms of resources when the number of frequent item sets is high. Eclat, on the other hand, is distinguished by its use of a vertical representation of the data and in-depth exploration, which generally makes it faster and more efficient, particularly when the dataset contains a large number of transactions and a limited number of items. These advantages make Eclat a more effective alternative for discovering frequent patterns.
So, although the choice of algorithm depends on the context and specific constraints of the data, Eclat appears to be a more suitable solution in the majority of cases requiring efficient association mining.